



Technisch verslag

Ondertekenportaal — wat de applicatie doet, hoe zij werkt, en
waarom de keuzes zo zijn gemaakt

Otto Visser & Partners accountants
26 juli 2026 · na changeset v1.7

Inhoud

1. Wat de applicatie is doel, gebruikers, omvang
2. De architectuur vijf containers en waarom
3. Technische stack alle onderdelen met versies
4. Het datamodel 14 modellen, 10 opsommingen
5. De ondertekenpipeline de kern van de applicatie
6. Digitale handtekening en tijdstempel PAdES, sealer, beroepscertificaat
7. Bewaartermijn en archivering doorvoerstation, niet archief
8. Het auditspoor hashketen, append-only, ankers
9. Beveiliging in lagen, met de grenzen erbij
10. E-mail en bezorging bounces, webhooks, herinneringen
11. Achtergrondtaken de wachtrij en tien taken
12. Documentherkenning tekstlaag, OCR, sjablonen
13. Gelijktijdigheid en vergrendeling waar het bijna stukging
14. Back-up en herstel wat het kroonjuweel is
15. Kwaliteit en tests 221 controles
16. Hosting en beheer 4 GB veilig maken
17. Wat bewust niet is gebouwd en waarom
18. Wat nog niet bewezen is eerlijk over de grenzen

HOOFDSTUK 1

Wat de applicatie is

Een zelf-gehost portaal waarmee Otto Visser & Partners documenten online laat ondertekenen. Cliënten hebben geen account: zij komen binnen via een eenmalige link en een verificatiecode. De accountant ondertekent met zijn beroepscertificaat, wat de natte handtekening vervangt.

1.1 Het uitgangspunt: een doorvoerstation

De belangrijkste ontwerpbeslissing is niet technisch maar organisatorisch. Dit portaal is **geen archief**. Een dossier wordt aangemaakt, de cliënt tekent, de accountant waarmerkt, het stuk gaat naar SharePoint, en daarna verdwijnt het uit het portaal. De bewaarplicht ligt bij SharePoint.

Daaruit volgt alles wat verder in dit verslag staat. Het primaire bewijs is de ondertekende PDF, niet de database: dat bestand bevat een gekwalificeerde handtekening met tijdstempel en ingebedde revocatiegegevens, plus een certificaatpagina met de instemmingstekst. Het is self-contained. Iemand kan het over vijf jaar in Acrobat openen en valideren zonder onze server, onze database of onze medewerking. Het auditspoor in de database is detail eromheen: maillevering, mislukte codes, IP per stap. Waardevol bij een betwisting, maar niet de drager van het bewijs.

1.2 Voor wie en hoeveel

Kantoorgebruikers	medewerkers en accountants, maximaal 2 gelijktijdig, op werkdagen
Cliëntkant	24/7 bereikbaar, want cliënten tekenen op hun eigen moment
Volume	ongeveer 10 ondertekende documenten per werkdag
Documenten in het portaal	90 dagen na archivering
Auditspoor in het portaal	7 jaar na afronding
Archief	SharePoint, handmatig gevuld, met een vinkje in het portaal
De handtekening	het beroepscertificaat van de accountant (AA of RA)

1.3 Wat de applicatie kan

- Word- en PDF-documenten importeren, met automatische conversie naar PDF
- Het documenttype herkennen (jaarrekening, notulen, akkoordverklaring...) en op basis daarvan een titel en begeleidend bericht voorstellen
- Meerdere documenten in één verzoek, of elk apart
- Tekenvelden plaatsen op de pagina, per ontvanger
- Ontvangers uit de eigen cliëntendatabase halen
- Versturen per e-mail met een eenmalige, kortlevende link
- Tekenen in de browser, na een verificatiecode per e-mail of sms
- Volgordelijk of gelijktijdig laten ondertekenen
- Weigeren met opgaaf van reden
- Automatische herinneringen, vervalttermijn en opnieuw versturen
- Gekwalificeerd ondertekenen door de accountant, met bevestiging per pincode
- Een ondertekencertificaat als extra pagina in het document
- Een ondertekend document controleren
- Cliënten beheren en importeren uit Excel of CSV
- Gebruikersbeheer met rollen en tweefactorauthenticatie
- Berichtsjablonen per documenttype beheren

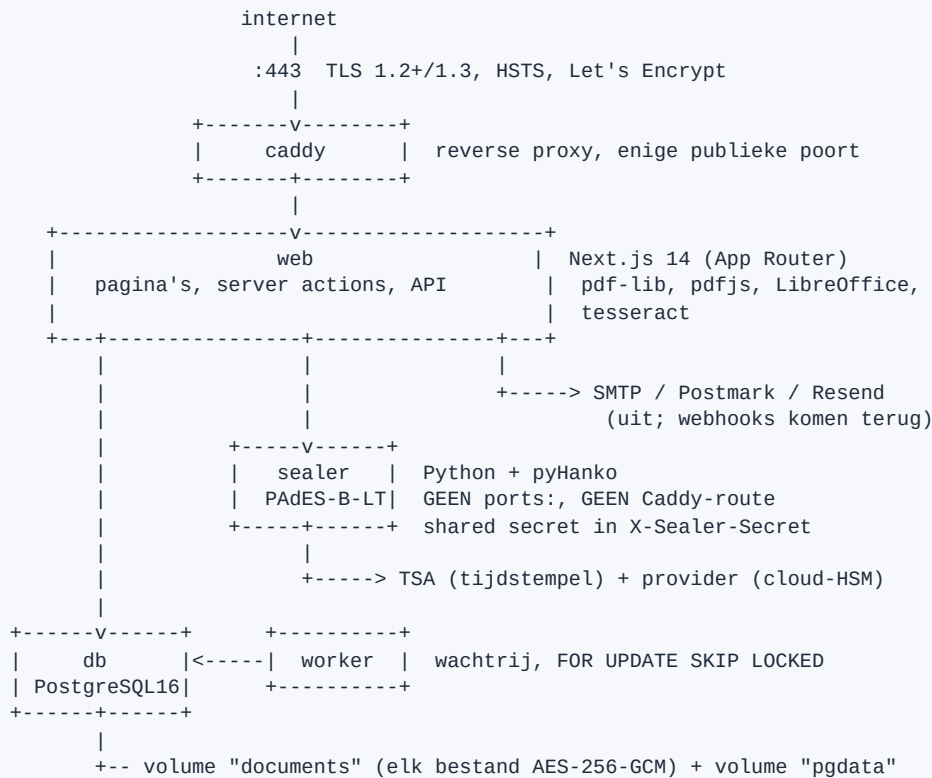
1.4 Omvang van de codebase

TypeScript/React	150 bestanden, ongeveer 17.500 regels
Python (de sealer)	ongeveer 790 regels
Databasemigraties	21
Datamodellen	14, met 10 opsommingstypen
Soorten auditgebeurtenissen	36
Omgevingsvariabelen	88, allemaal gevalideerd bij het opstarten
Automatische controles	221 (175 applicatie, 26 sealer, 16 certificering, 4 validatormodus)

HOOFDSTUK 2

De architectuur

Vijf containers op één VPS. Elke scheiding heeft een reden; containers die er niets toe deden zijn er weer uit gehaald.



2.1 Waarom deze verdeling

De sealer apart

pyHanko is Python en heeft de LT-laag ingebouwd: revocatiegegevens in de DSS-dictionary, een document-timestamp, en correcte incrementele updates. In pdf-lib is dat handwerk en foutgevoelig. Bijkomend voordeel: de container die bij de ondertekengegevens kan, is niet publiek bereikbaar — geen ports: -blok en geen route in Caddy.

De worker apart

Een verzegeling die faalt moet opnieuw kunnen zonder dat er een HTTP-verzoek open staat. Ook herinneringen, vervaltermijnen en opruimtaken horen niet in een webverzoek. Eén instance: de wachtrij kan meer aan, maar de volgorde-garanties zijn daar niet op ontworpen.

De database niet publiek

Alleen bereikbaar op het interne Docker-netwerk. De firewall op de host laat alleen 22, 80 en 443 door.

2.2 Wat er niet is, en waarom niet

Er was een zesde container gebouwd: een aparte *validator* voor de controlepagina, omdat een publieke upload die door een PDF-parser gaat niet in dezelfde container hoort als de ondertekengegevens. Die is er weer uit. De pagina staat nu achter de login, en daarmee is er geen onbeauthenticerde uploadingang meer. Dat is dezelfde bescherming voor minder techniek, en het scheelt geheugen op een machine van 4 GB.

De rol bestaat nog in de image: met `VALIDATOR_ONLY=true` weigert het sealer-proces te starten als er ondertekengegevens in zijn omgeving staan, en geven `/seal`, `/prepare` en `/inject` een 403. Wordt de pagina ooit publiek, dan is dat de weg terug; de compose-snippet staat in de hostinghandleiding.

Een grendel in het verkeerde proces is geen grendel. Die controle stond eerst in de omgevingsvalidatie van Next.js. De validator-container draait het Python-proces, dus die code liep daar nooit. Nu staat hij in `sealer/main.py`, waar hij thuishoort.

Technische stack

3.1 Applicatie

Onderdeel	Versie	Waarvoor
Next.js	14.2	App Router, Node-runtime (nodig voor pdf-lib, LibreOffice, nodemailer)
React	18.3	server components met losse client-eilanden
TypeScript	5.x	strikt; <code>tsc --noEmit</code> is een poort in de CI
Tailwind	3.4	opmaak
PostgreSQL	16	database, wachtrij, rate-limiting, advisory locks
Prisma	5.22	schema, migraties, typeveilige queries
@cantoo/pdf-lib	2.7	stempelen, auditcertificaat, plat slaan
pdfjs-dist	4.8	pagina's tonen bij het plaatsen van velden; tekst uitlezen
@node-rs/argon2	2.0	wachtwoorden met argon2id
otpauth + qrcode	9.3 / 1.5	twefactorauthenticatie (TOTP)
nodemailer	9.0	SMTP; Postmark en Resend via hun REST-API
zod	3.23	validatie op elke grens, ook op de omgeving
rate-limiter-flexible	5.0	limieten, met de tellers in Postgres
mammoth	1.12	terugval voor .docx als LibreOffice ontbreekt
@e965/xlsx	0.20	cliënten importeren uit Excel
nanoid	5.1	opslagsleutels die nooit botsen
pg	8.22	directe SQL voor rate-limiting en advisory locks

3.2 De sealer

Onderdeel	Versie	Waarvoor
FastAPI + uvicorn	0.115 / 0.34	vijf eindpunten, geen documentatieroutes
pyHanko	0.25.3	PAdES-B-LT: handtekening, tijdstempel, revocatiegegevens
pyhanko-certvalidator	0.26.5	validatie van ketens en revocatie
requests	2.32	naar de signing-API en de TSA

3.3 Buiten de containers

- **LibreOffice headless** in de web-image: Word en Excel naar PDF.
- **tesseract** met Nederlandse taaldata plus **poppler**: OCR als een PDF geen tekstlaag heeft.
- **Caddy 2**: automatische HTTPS met Let's Encrypt.
- **restic** op de host: versleutelde back-up naar externe opslag.

3.4 Wat de applicatie niet nodig heeft

Geen Redis (de wachtrij en de limieten staan in Postgres), geen aparte broker, geen zoekmachine, geen S3 (de opslagabstractie staat er wel klaar voor), geen externe authenticatiedienst. Dat is bewust: elk extra onderdeel is een extra ding dat kan omvallen en dat iemand moet bijhouden.

HOOFDSTUK 4

Het datamodel

Veertien modellen. De interessante velden zijn niet de namen en adressen, maar de velden die bewijs vastleggen.

4.1 De modellen

Model	Rol
Accountant	medewerker van het kantoor, met rol, tweefactor en eventueel een beroepscertificaat
Session	intrekbare serversessie; alleen de hash van het token
Client	cliëntendatabase, met klantnummer en archiefmap
Dossier	één ondertekenverzoek met een of meer documenten
Document	één te ondertekenen stuk, met vijf opslagsleutels en hun hashes
Recipient	één ondertekenaar, intern of extern, met alle bewijsvelden
SignatureField	een tekenvak: pagina en coördinaten in PDF-punten
AuditEvent	append-only gebeurtenis, met hashketen per dossier
AuditAnchor	de kop van een keten, buiten de database vastgelegd
CscSigningSession	ondertekensessie die een browserredirect moet overleven
MailEvent	ruwe bezorggebeurtenis van de mailprovider, idempotent
Job	de wachtrij
MessageTemplate	berichtsjablonen per documenttype
RateLimit	tellers van de rate-limiter (beheerd door de bibliotheek zelf)

4.2 De opsommingstypen

Type	Waarden
DossierStatus	CONCEPT, VERZONDEN, GEDEELTELIJK, ONDERTEKEND, WACHT_OP_WAARMERK, SEALING_FAILED, GEWEIGERD, VERLOPEN
SealStage	NONE, PRESEAL, QUALIFIED, SEALED
PartyStatus	PENDING, SIGNED, DECLINED
SigningMode	PARALLEL, SEQUENTIAL
MailStatus	QUEUED, SENT, DELIVERED, BOUNCED, COMPLAINED, FAILED
DocumentKind	JAARREKENING, NOTULEN_AVA, BEVESTIGING_JAARREKENING, AKKOORD_IB, AKKOORD_VPB, OPDRACHTBEVESTIGING, OVERIG
VerificationMethod	EMAIL, SMS
Role	MEDEWERKER, BEHEERDER
PartyRole	ZELF (kantoor), EXTERN (cliënt)
FieldKind	SIGNATURE, DATE, INITIALS

4.3 De velden die bewijs vastleggen

Op Recipient

tokenHash	SHA-256 van de tekenlink. De link zelf staat nergens; hij is niet opnieuw te versturen, alleen te vervangen.
otpHash , otpAttempts , otpVerifiedAt	de verificatiecode, alleen gehasht, met pogingenteller
consentTextSnapshot + hash + consentShownAt	de <i>letterlijke</i> tekst die deze persoon op het scherm zag, op dat moment. Niet de huidige tekst.
presentedHashes	per document de SHA-256 van de bytes die deze persoon te zien kreeg
mailStatus , mailBounceReason	of de uitnodiging is aangekomen

Op Document

Vijf opslagsleutels, elk met een eigen hash. Dat zijn geen versies van het gemak maar herstartpunten:

originalKey	zoals geüpload (na eventuele conversie naar PDF)
workingKey	met de zichtbare stempels tot nu toe
preSealKey + preSealSha256	na het auditcertificaat en het plat slaan — herstartpunt 1
postQualifiedKey + hash	na de gekwalificeerde handtekening — herstartpunt 2
sealedKey + sealedSha256	het definitieve stuk — herstartpunt 3
timestampedAt , sealCertSerial , sealTsaUrl	de bewijstijd uit de tijdstempeldienst en welk certificaat is gebruikt
sealStage	hoe ver het is, expliciet en niet afgeleid uit gevulde sleutels

`sealStage` is er gekomen omdat "afleiden uit welke sleutels gevuld zijn" precies de fout is die twee keer verzegelen mogelijk maakte. Valt het proces om tussen het wegschrijven van de bytes en het committen van de rij, dan is het bestand verzegeld en weet de database dat niet.

Op `AuditEvent`

`prevHash` en `hash` vormen de keten, `seq` is een oplopend nummer, `metadata` is JSON die wordt *gecanonicaliseerd* voordat hij in de hash gaat — Postgres bewaart de sleutelvolgorde van JSONB niet, dus zonder canonicalisatie zou de keten na teruglezen onterecht gebroken lijken.

HOOFDSTUK 5

De ondertekenpipeline

Dit is de kern. De volgorde is strikt, en de reden is onvermijdelijk: zodra er een cryptografische handtekening in een PDF zit, mag het bestand niet meer worden bewerkt.

```
CONCEPT
| importeren: PDF of Word -> LibreOffice -> PDF
| herkennen: tekstlaag -> trefwoorden -> boekjaar -> titel + bericht
| velden plaatsen (pixels -> PDF-punten), ontvangers kiezen
v
VERZONDEN -----> per ontvanger: eenmalig token (alleen de hash bewaard)
|                     + uitnodigingsmail met de link
|
| cliënt opent -> verificatiecode -> tekent in de browser
v
GEDEELTELIJK ---> stempel per ondertekenaar, binnen een rijvergrendeling
|
| ALLE PARTIJEN KLAAR
v
1. zichtbare handtekeningen en stempels staan erop
2. label "Digitaal ondertekend door: , "
3. ondertekencertificaat als extra pagina erachter
4. plat slaan
5. preSealSha256 vastleggen, artefact apart opslaan --> sealStage = PRESEAL
|
v
WACHT_OP_WAARMERK
| de accountant selecteert de stukken en bevestigt ÉÉN keer met zijn pincode
| (tot 50 hashes onder één bevestiging)
v
6. gekwalificeerde handtekening injecteren --> sealStage = SEALED
   sealedKey, sealedSha256, timestampedAt, sealCertSerial
|
v
ONDERTEKEND ---> voltooiingsmail met bijlage, en het archiveervinkje verschijnt
```

5.1 Waarom het certificaat vóór de handtekening komt

Het ondertekencertificaat en het plat slaan zijn bewerkingen. Die moeten dus gebeuren voordat er een handtekening in het bestand zit. Maar ze kunnen pas als alle partijen hebben getekend — en op dat moment is de accountant er niet noodzakelijk bij om een pincode in te voeren. Vandaar de wachtstand `WACHT_OP_WAARMERK`: geen fout, een normale stap.

5.2 Fail-closed

Lukt de verzegeling niet, dan komt het dossier op `SEALING_FAILED`. Er gaat **geen voltooiingsmail** uit, er staat een melding op het dashboard met de laatste foutmelding, en een taak probeert het opnieuw na 1, 5, 15 en 60 minuten en daarna elk uur, maximaal twaalf keer. Na drie mislukte pogingen krijgt de eigenaar een e-mail. Een fout die pyHanko structureel weigert (een 400 in plaats van een 502) stopt direct met een melding, in plaats van eeuwig rond te draaien.

Voor de cliënt verandert er niets: die heeft geldig ondertekend, alleen de afronding wacht.

5.3 Wat er per ondertekenaar wordt vastgelegd

Op het moment dat iemand tekent, staat vast: het IP-adres, de user-agent, het tijdstip van de geverifieerde code, het tijdstip van ondertekenen, de letterlijke instemmingstekst die op het scherm stond, en de hash van de bytes die déze persoon te zien kreeg. Dat laatste is belangrijk bij ondertekenen op volgorde: de tweede ondertekenaar ziet een ander bestand dan de eerste, en dat is geen aanwijzing dat

er is gerommeld. Het ondertekencertificaat legt dat ook uit, zodat niemand die hashes met het eindbestand gaat vergelijken en de verkeerde conclusie trekt.

5.4 De accountant is een gewone ontvanger

Een kantoorondertekenaar heeft net als een cliënt een positie in de volgorde. Het afronden begint zodra *alle* ontvangers hebben getekend, ongeacht rol. Het verschil zit alleen in de authenticatie: een cliënt komt binnen via een magic link plus een code per e-mail, een medewerker via inloggen op `/te-ondertekenen`.

Dat kan omdat **iedere handtekening van iedere partij een visuele stempel is**. Alleen het zegel aan het eind is cryptografisch. De volgorde is daarom vrij configureerbaar zonder cryptografische gevolgen — geen toeval, maar de reden dat deze opzet werkt.

De zwakke plek, en wat eraan is gedaan. Een sessie leeft uren. Wie langsloopt bij een onvergrendelde laptop zou kunnen ondertekenen namens degene die is ingelogd. Daarom vraagt het indienen van een kantoorondertekening om een **verse code uit de authenticatie-app**. Bewust niet om het wachtwoord: dat vullen browsers in, en dan bewijs je niets over het moment.

Het detail dat hierbij het vaakst wordt overgeslagen: een TOTP-code blijft binnen zijn tijdvenster geldig en is dus tot anderhalve minuut herbruikbaar. De laatst geaccepteerde tijdstap wordt daarom vastgelegd en niet nogmaals geaccepteerd — ook niet als hij bij het inloggen is gebruikt. Dat is het verschil tussen een bewijs van bezit en een bewijs van kopiëren.

Vijf misslagen blokkeren die ondertekening, niet het account: anders sluit een accountant zich met een typefout uit de hele applicatie. Vakantiedekking gaat via een expliciete **overdracht** aan een collega, met een auditregel die vastlegt van wie naar wie en door wie — niet via “iedereen mag elk kantoorveld invullen”.

5.5 Het getekende exemplaar

De voltooiingsmail is meer dan een bevestiging; hij is het eigen exemplaar van de cliënt. Daarom staat de SHA-256 van het verzegelde bestand als leesbare tekst in de body. Die belandt zo in zijn mailbox, met zijn ontvangstdatum, buiten ons beheer — en dat dekt precies het scénario waarin een onafhankelijk zegel telt: dat iemand *ons* verdenkt. Wij kunnen die mail niet aanpassen.

Daarnaast zit er een downloadlink bij, met een eigen token. Bewust gescheiden van het tekentoken: dat is eenmalig en verbruikt na ondertekening, en dat moet zo blijven. Het downloadtoken is herbruikbaar binnen zijn termijn (90 dagen, gelijk aan hoelang de bestanden in het portaal staan), en kan niet worden gebruikt om te ondertekenen — andere route, andere validatie. Elke download komt als `GEDOWNLOAD` in het auditspoor. Zodra de bestanden worden opgeruimd, wordt het token gewist en geeft de link een uitgelegde melding in plaats van een blanco foutpagina.

5.6 Het ondertekencertificaat

Een extra pagina achter het document, in het Nederlands. Het uitgangspunt: iemand die over vijf jaar *alleen dit PDF-bestand* in handen krijgt, zonder toegang tot het portaal, moet het hele verloop kunnen nalopen. Alles wat daarvoor nodig is staat dus op het blad en niet alleen in de database.

Blok	Wat er staat
Document	titel, dossierkenmerk, omvang in pagina's en het aantal handtekening-, paraaf- en datumvelden. Zo beschrijft het certificaat aantoonbaar het stuk waar het aan vastzit.
Per ondertekenaar	naam en e-mail, wanneer de uitnodiging is verstuurd, wanneer het stuk voor het eerst is geopend, wanneer de verificatiecode is ingevoerd en wanneer er is getekend — plus IP-adres, een leesbare samenvatting van het apparaat en de volledige user-agent.
Identiteitscontrole	in gewone taal welke controle is uitgevoerd: e-mailcode, sms-code, of ingelogde kantoormedewerker met tweefactorauthenticatie.
Instemming	de letterlijke tekst die deze persoon op het scherm las, met het tijdstip. Een later gewijzigd sjabloon verandert niet wat er is gelezen.
Integriteit	de SHA-256 van de ondertekende inhoud, plus per ondertekenaar de hash van de versie die die persoon zag.

Alle tijden staan in `Europe/Amsterdam`, met de zomer- of wintertijd zoals die op dat moment gold. Dat staat er met zoveel woorden bij: een kaal “12:24:59” is onbruikbaar zodra iemand het naast een logregel uit een ander systeem legt.

Staat de verzegeling uit, dan staat er **“Dit document is niet verzegeld.”** Die regel is niet te onderdrukken, en het opmaken reserveert bewust ruimte zodat hij nooit los op een volgende pagina belandt, weg van de vingerafdruk waar hij over gaat.

Wat hier fout ging. Het certificaat drukte IP-adres en apparaat al af, maar kreeg ze nooit aangeleverd: er stond bij iedere ondertekenaar een streepje. De gegevens zaten wél in het auditspoor. Het viel niemand op, want er stond gewoon iéts. Ze worden nu bij het opmaken uit het auditspoor gehaald — één bron van waarheid — en een regressietest controleert de inhoud van het certificaat, niet alleen het bestaan ervan.

HOOFDSTUK 6

Digitale handtekening en tijdstempel

In een gewoon dossier is er één cryptografische handtekening: het organisatiezegel. Het beroepscertificaat is de uitzondering.

6.1 De drie standen

SEAL_MODE	Betekenis
none	geen cryptografische handtekening. Banner in het portaal, een niet-onderdrukbare regel op het certificaat, en <code>VERZEGELING_OVERGESLAGEN</code> in het auditspoor. In productie ook <code>ALLOW_UNSEALED=true</code> nodig: één vlag zet iemand per ongeluk, twee niet.
organisation	route A, de normale route. Het organisatiezegel is de enige cryptografische handtekening: certificerend met DocMDP P=1, onbeheerd en onzichtbaar. Zet <code>sealStage = SEALED</code> .
qualified	route B, de uitzondering. Alleen voor documentsoorten waar een beroepscertificaat vereist is: een SBR-accountantsverklaring of het waarmerken van een SBR-jaarrekening. De accountant autoriseert zelf.

De opstartgrendel. Met `SEAL_MODE=qualified` en `PROFESSIONAL_SIGNING_DRIVER=none` weigert de applicatie te starten. Dat is de gevaarlijkste stille toestand die er is: er wordt dan niets verzegeld terwijl de configuratie zegt van wel, en niets in de status verradt het.

6.2 Wat het certificerende zegel wel en niet doet

Het zegel certificeert met **DocMDP P=1** (`NO_CHANGES`). Dat is sterker dan een gewone handtekening: die zegt alleen “dit was de inhoud”, terwijl een certificerende handtekening ook zegt wat er daarna *nóg* mag. Bij P=1 is dat niets.

De ene echte onzekerheid vooraf was of de revocatiegegevens van de LT-laag de eigen permissie zouden schenden: die DSS-dictionary komt namelijk in een *látère* revisie dan de handtekening.

Nagemeten in `scripts/regressie/certificering.py`:

- Ja, de DSS komt in een latere revisie — en dat kan niet anders. De DSS bevat validatie-informatie over de handtekening, dus die moet er al zijn. Er is geen variant van PAdES B-LT waarin het anders ligt.
- Nee, dat schendt de permissie niet. De PAdES-regels zonderen DSS-updates expliciet uit, en pyHanko meldt `docmdp_ok=true` met “acceptable modifications”.
- Een echte inhoudelijke wijziging *wórdt* gemeld, en niet als “er is iets veranderd” maar als permissieschending. Dat onderscheid is precies wat certificeren toevoegt.
- Sterker dan verwacht: onder P=1 *weigert* pyHanko een tweede handtekening te zetten. Niet “het mag maar het wordt gemeld” — het gereedschap doet het niet.

Wat we hierover niet beweren. Geen enkel PDF-mechanisme verhindert fysiek dat iemand bytes achter een bestand plakt; dat is in dezelfde test aangetoond en geldt bij elke leverancier. En het gedrag verschilt per viewer. De formulering is daarom niet “het document kan niet worden gewijzigd” maar **“wijzigen wordt geblokkeerd en elke wijziging is aantoonbaar”**. Wat niet is getest: Adobe Acrobat zelf — daar bestaat geen headless variant van. Dat hoort met de hand te gebeuren zodra er een echt certificaat is.

`PADES_LEVEL=lt` is hierbij een eis en geen voorkeur. Bij LTA komt er een losse document-timestamp als incrementele update bovenop, en dat is precies de combinatie die met certificeren wringt. De sealer weigert die combinatie.

6.3 Waarom geen tweede zegel eróverheen

Dat stond er wel, met een eigen driver voor GlobalSign. Het is eruit. Het beroepscertificaat doet hetzelfde werk: het beschermt de integriteit én het vervangt de natte handtekening. Twee mechanismen naast elkaar betekent een tweede certificaat (€450–650 per jaar), een tweede storingsbron, en de vraag welke van de twee er nu eigenlijk toe doet.

6.4 En waarom er ook geen tweede ondertekendriver meer is

Er was een driver (`digidentity`) die tijdens het ondertekenen namens de accountant tekende, zonder handeling van hem. Prettig in gebruik, maar hij tekende **best-effort**: viel de provider weg, dan bleef het zichtbare stempel staan, kwam er een auditregel “mislukt: ...” en liep het dossier door naar ONDERTEKEND. Het resultaat was een stuk dat ondertekend oogt zonder gekwalificeerde handtekening, en niemand kreeg bericht.

Dat is de verkeerde faalrichting op precies het punt waar je op vertrouwt. De Cleverbase-route faalt juist dicht: kan er niet gewaarmerkt worden, dan gaat het dossier naar `SEALING_FAILED` en de deur niet uit. De driver is verwijderd en de applicatie weigert te starten als hij nog in het `.env`-bestand staat, met een melding die zegt wat er in de plaats moet.

Waarom niet gewoon de fout doorgeven. Dat was de andere optie: het ondertekenen laten klappen als de provider wegvalt. Maar dan verliest de cliënt zijn handtekening door een storing bij een derde partij, terwijl hij er zelf niets aan kan doen. Het waarmerken hóort dus niet in die stap thuis — het gebeurt later, apart, waar mislukken alleen het kantoor raakt en het dossier netjes blijft wachten.

6.5 De vijf eindpunten van de sealer

Eindpunt	Wat het doet
<code>/health</code>	kan de driver worden gebouwd, is de TSA bereikbaar
<code>/seal</code>	één PAdES-handtekening in één keer (nu ongebruikt: het pad zonder gebruikersautorisatie)
<code>/prepare</code>	fase 1: placeholder met correcte ByteRange, geeft de te ondertekenen hash terug
<code>/inject</code>	fase 2: bouwt de CMS met de handtekening van de provider en zet die in de voorbereide PDF
<code>/validate</code>	controleert de handtekening(en); slaat niets op

6.6 Tweefasig ondertekenen, en waarom dat nodig is

Bij Cleverbase autoriseert de *accountant*, niet de server: hij bevestigt met een pincode in de app van de provider, na een browserredirect. Daardoor moet het ondertekenen in twee delen:

1. **Voorbereiden.** Voor *alle* geselecteerde documenten wordt een placeholder gezet en de hash berekend. Dat moet vóór de autorisatie, want het autorisatietoken (de SAD) leeft maar 300 seconden en dekt de hele verzameling in één keer.
2. **Injecteren.** Na de bevestiging komen de handtekeningen terug en worden ze in de voorbereide bestanden gezet.

De sessie moet die redirect overleven. Daarvoor is `CscSigningSession` : een `state` van 32 willekeurige bytes (de CSRF-bescherming op de callback), de voorbereide bestanden versleuteld op schijf, en de hashes zoals ze zijn verstuurd.

De invariant die een hele klasse fouten vangt

Tussen voorbereiden en injecteren zit een menselijke pauze van minuten. Zit er ook maar één tijdsafhankelijk gegeven in de te ondertekenen structuur, dan verandert de hash en is de handtekening stil ongeldig. Daarom worden de verstuurde hashes apart bewaard en vóór het injecteren byte-voor-byte vergeleken. Wijken ze af, dan is dat een programmeerfout en geen storing: de sessie gaat op `FAILED`, er komt geen nieuwe poging, en de beheerder krijgt bericht. Een retry zou de fout alleen maskeren.

Drie soorten fouten

Categorie	Voorbeelden	Gedrag
opnieuw zonder pincode	injectiefout, opslagfout na een geslaagde ondertekening	automatische nieuwe poging
opnieuw met pincode	autorisatietimeout, gebruiker weigert, token verlopen	sessie <code>EXPIRED</code> , accountant begint opnieuw
nooit opnieuw	hash-mismatch, verkeerd aantal handtekeningen, onbekend credential	sessie <code>FAILED</code> , melding

De verzameling is **atomair**: mislukt er één, dan wordt er niets geïnjecteerd. Een half ondertekende verzameling is erger dan geen.

6.7 De PDF bepaalt of er al ondertekend is, nooit de database

Tussen het wegschrijven van de bytes en het committen van de databaserij kan het proces omvallen. De rij weet dan niet dat er al is ondertekend, een nieuwe poging ondertekent opnieuw, en er staan twee handtekeningen in één document. Niet ongeldig, maar onuitlegbaar op een auditcertificaat.

Daarom inspecteren `/seal`, `/prepare` en `/inject` het aangeleverde bestand op een bestaande handtekening in het gevraagde veld. Is die er, dan geeft de sealer een `409` met serienummer en tijdstip, en tekent hij niet. De aanroeper behandelt dat als succes en werkt alleen de administratie bij.

Bij `/inject` was de vraag of dit zin heeft: die krijgt per definitie een bestand met een lege placeholder. Nagemeten: pyHanko ziet een placeholder niet als handtekening, dus de controle is wel zinvol. Dat is precies het soort ding dat je niet moet beredeneren maar meten.

6.8 De zichtbare tekst per documentsoort

Documentsoort	Wat er in de handtekening staat
<code>JAARREKENING</code>	Ondertekend door <naam>, AA of RA
notulen, bevestiging, opdrachtbevestiging, overig	Verzegeld door Otto Visser & Partners Accountants
akkoordverklaringen IB en VPB	nu “verzegeld”; instelbaar

De reden voor de splitsing: een gekwalificeerde handtekening draagt juridisch gewicht. Op een akkoordbrief waarin de *cliënt* iets verklaart, wil je niet de indruk wekken dat de accountant die verklaring

mede onderschrijft. Daar is de handtekening alleen het slot op het document. Dit is een instelling (`SIGN_AS_AUTHOR_KINDS`) en geen code, zodat het antwoord op de vraag “tekenen we op het aangiftestuk zelf of alleen op de akkoordbrief?” later kan komen.

6.9 De controlepagina

Onder **Document controleren** kan een medewerker een ondertekend stuk uploaden. De pagina geeft **één eindoordeel** met drie toestanden:

Toestand	Wat er staat
groen	Geldig en uitgever bevestigd
oranje	Ongewijzigd, maar uitgever niet te verifiëren — met de waarschuwing dat dit ook past bij een document dat door iemand anders opnieuw is ondertekend
rood	Gewijzigd of ongeldig

Waarom één oordeel en niet twee vinkjes: zonder vertrouwensketen staat alleen vast dat de bytes niet zijn gewijzigd en dat de ondertekenaar de sleutel had bij het certificaat dat *in het document zelf* zit. Iemand kan een jaarrekening pakken, een bedrag wijzigen en opnieuw ondertekenen met een zelfgemaakt certificaat waarin “Otto Visser & Partners” als naam staat. De integriteitscontrole zegt dan groen. Met twee losse vinkjes leest een medewerker van een bank het vinkje en stopt. Daarom staan subject én issuer letterlijk op de pagina, met een label als het certificaat zelfondertekend is.

De pagina haalt **niets van internet**. Zou zij revocatiegegevens ophalen, dan bepaalt de geüploade PDF welke adressen de server benadert — een aanvaller kan dan naar interne adressen laten bellen. Voor een document met ingebedde revocatiegegevens is dat ook geen verlies: die zitten er al in.

HOOFDSTUK 7

Bewaartermijn en archivering

Twee termijnen, en één regel die het veilig maakt.

Wat	Termijn
Documentbestanden (alle vijf de opslagsleutels)	90 dagen na het archiveervinkje
Auditspoor, hashketen, metadata, bewijsvelden van ontvangers	7 jaar na afronding

Nooit op de klok verwijderen, altijd op de vlag. Automatisch verwijderen naast handmatig archiveren is de enige echt gevaarlijke combinatie in dit ontwerp. Vergeet iemand af te vinken en verwijdert de klok toch, dan is het stuk nergens meer. Daarom verdwijnt er van een afgerond dossier *niets* zolang het vinkje uit staat — desnoods voor altijd. Vollopen is een acceptabel probleem, verlies niet.

7.1 De werkwijze

1. De medewerker downloadt het ondertekende stuk op de dossierpagina.
2. Hij zet het in de klantmap in SharePoint.
3. Hij vinkt het af in het portaal, met de map erbij. Dat komt in het auditspoor: wie, wanneer, waar.
4. Negentig dagen later verdwijnen de bestanden. De administratie blijft.

Zonder signalering zou dit een geheugenspel zijn. Daarom: een teller “**wacht op archivering**” op het dashboard met de oudste bovenaan, een bericht naar de eigenaar op dag 60, en op dag 80 naar de eigenaar én de beheerders — met de eerlijke tekst dat er niets automatisch gebeurt, maar dat het daarna handmatig moet worden opgeschoond.

7.2 Wat er wél zonder vinkje weggaat

Dossiers die nooit zijn afgerond: verlopen, geweigerd, en concepten die nooit zijn verstuurd. Na 90 dagen gaan daar de bestanden weg zonder vinkje. Er is geen ondertekend stuk om te bewaren. Het auditspoor blijft ook daar zeven jaar.

7.3 Wat er precies gebeurt bij het opruimen

Eerst de administratie: de vijf sleutelvelden worden leeggemaakt. Daarna de bestanden. Die volgorde is niet willekeurig. Blijft er een bestand staan waar niets meer naar wijst, dan is dat een wees en die wordt later opgeruimd — hinderlijk, niet gevaarlijk. Andersom (bestand weg, sleutel nog gevuld) is dataverlies.

Er komt een grafsteen in het auditspoor met het aantal gewiste bestanden per sleuteltype, en met de vermelding dat het bewijs blijft. Na zeven jaar verdwijnt het hele dossier inclusief auditspoor, met een tweede grafsteen in de dossierloze reeks die het aantal regels en de laatste hash van de opgeruimde keten vastlegt.

7.4 Twee afspraken buiten de applicatie

- **Bewerk het PDF niet in SharePoint.** Uploaden laat de bytes intact, maar wie in de ingebouwde editor een aantekening maakt en opslaat, breekt de handtekening. Richt de bibliotheek zo in dat bewerken niet de gewone route is.
- **Vraag na of er back-up op SharePoint zit.** De prullenbak is 93 dagen en versiebeheer is geen back-up. Licht hier zeven jaar aan ondertekende jaarrekeningen als enige kopie, dan hoort daar iets als M365

Backup bij.

Steekproef: haal af en toe een gearriveerd stuk uit SharePoint en gooi het door de eigen controlepagina. Blijft dat groen, dan klopt de keten.

HOOFDSTUK 8

Het auditspoor

36 soorten gebeurtenissen, append-only afgedwongen door de database, met een hashketen per dossier.

8.1 Wat er wordt vastgelegd

Aanmaken, versturen, openen, code verstuurd, code geverifieerd, code mislukt, code geblokkeerd, ondertekend, gekwalificeerd ondertekend, geweigerd, herinnerd, opnieuw verstuurd, verlopen, verzegeld, verzegeling mislukt, verzegeling overgeslagen, integriteitsafwijking, gedownload, gearchieveerd, ingetrokken, ingelogd, certificaat ingetrokken, de vier stappen van de ondertekensessie, de vier mailstatussen, en de grafsteen van de bewaartermijn.

8.2 Append-only, en wat dat wel en niet betekent

Een trigger in de database weigert elke `UPDATE` en `DELETE` op de tabel. Niet als afspraak, maar als mechanisme. Alleen de geplande opruiming zet die kortstondig uit, binnen één transactie.

De grens, expliciet. Die trigger beschermt tegen een fout in de applicatie en tegen iemand die alleen databasetoegang heeft en niet van de noodschakelaar weet. Hij beschermt *niet* tegen een beheerder met volledige rechten: die kan dezelfde schakelaar zetten. Het auditspoor is dus aantoonbaar onaangetast tegenover *de applicatie*, niet tegenover *de databasebeheerder*.

8.3 De hashketen loopt per dossier

Elke regel bevat de hash van de vorige. Niet één globale keten, en dat is bewust: de bewaartermijn ruimt een compleet dossier op, en bij een globale keten zou zo'n legitieme opruiming de keten van alle andere dossiers breken. Een keten die altijd “gebroken” is, wordt genegeerd en beschermt dus niets. Regels zonder dossier (inloggen, grafstenen) vormen samen één eigen keten.

Wat de controle aantoont

- elke wijziging van een bestaande regel;
- het verwijderen van een regel middenin een keten;
- het afknippen van de kop van een keten — via één invariant: de eerste regel van een keten heeft geen voorganger. Knijpt iemand de kop eraf, dan begint de keten met een regel die naar een hash verwijst die niet meer bestaat.

Wat zij niet aantoont

Het verwijderen van een compleet dossier mét zijn hele keten. Daarvoor is een anker buiten de database nodig.

8.4 De ankers

Bij het afronden van een dossier, en dagelijks voor de lopende zaken, wordt de kop van de keten buiten de database vastgelegd: dossiernummer, laatste volgnummer, laatste hash, tijdstip. Naar twee bestemmingen — het archief én een postbus van het kantoor — want het archief is best-effort en mail kan stil falen. Met één bestemming heb je geen anker maar de illusie ervan. Komt er drie dagen achter elkaar niets aan, dan krijgen de beheerders bericht.

Wat een anker *niet* dekt: een herschrijving van gebeurtenissen *tussen* twee ankers blijft onzichtbaar. Dat venster sluit alleen een sleutelgebaseerde keten (HMAC). Het anker bij afronding sluit het venster wel voor het bewijs dat telt, want op dat moment is het dossier definitief.

8.5 Twee standen bij het schrijven

Voor de meeste soorten mag het auditspoor de flow niet blokkeren: een mislukte “gedownload”-regel is hinderlijk, geen ramp. Voor vijf soorten geldt het omgekeerde — ondertekend, gekwalificeerd ondertekend, code geverifieerd, verzegeld, geweigerd. Daar is de auditregel het bewijs zelf, en een handtekening zonder spoor is precies het scenario waarvoor het auditspoor bestaat. Die regels gooien door en rollen de hele indiening terug: liever een mislukte indiening die de cliënt kan herhalen dan een handtekening zonder bewijs.

De lijst blijft kort. Elke toevoeging maakt het ondertekenen afhankelijk van het auditpad, en dat is een beschikbaarheidsrisico.

HOOFDSTUK 9

Beveiliging

In lagen, met bij elke laag de grens erbij. Een maatregel waarvan je denkt dat hij meer doet dan hij doet, is gevaarlijker dan geen maatregel.

9.1 Onderweg

TLS 1.2+ via Caddy met automatische certificaatvernieuwing, HSTS, HTTP naar HTTPS. Cookies met `HttpOnly`, `Secure` en `SameSite`. Een Content-Security-Policy met een nonce per verzoek, plus `nosniff`, `frame-ancestors 'none'`, en een restrictief referrer- en permissionsbeleid. E-mail naar Microsoft 365 met STARTTLS afgedwongen.

9.2 In rust

Elk document is met **AES-256-GCM** versleuteld voordat het het volume raakt, met envelope-encryptie: per bestand een eigen datasleutel en een eigen IV, ingepakt onder een mastersleutel die alleen via de omgeving in het geheugen komt. De GCM-authenticatietag detecteert manipulatie. Tweefactorsleutels zijn kolom-versleuteld. Wachtwoorden zijn argon2id-hashes; codes en tekenlinks worden *alleen* gehasht bewaard.

Sleutelrotatie

De sleutelversie staat **in de data zelf** (een header in het bestand, een prefix in een versleutelde string), niet in een kolom. Ontsleutelen gebeurt met de versie uit de data, versleutelen met de huidige. Oude data blijft dus leesbaar terwijl nieuwe data een nieuwe sleutel gebruikt, en roteren kan zonder downtime. Ontbreekt een sleutel waar nog data naar verwijst, dan komt er een expliciete melding met het versienummer in plaats van een cryptische fout.

9.3 Toegang

Kantoorgebruikers	wachtwoord (argon2id) + verplichte TOTP met herstelcodes
Sessies	server-side en intrekbaar, met inactiviteits- en absolute limiet
Brute force	oplopende blokkade: 5 pogingen → 5 min, 8 → 30 min, 12 → 2 uur; gehard tegen het aftasten van bestaande e-mailadressen
Autorisatie	server-side op elke route; ondoorzichtige id's plus eigenaarscontrole
Cliënten	geen account: een eenmalige, kortlevende, gehashte tekenlink plus een verificatiecode per e-mail of sms
Limieten	op inloggen, codes, tekenlinks, wachtwoordherstel en de controlepagina; tellers in Postgres met een in-memory achtervang, zodat een databasestoring geen brute-force-venster opent

De erkende beperking. De tweede factor gaat naar hetzelfde e-mailadres als de link. De koppeling tussen handtekening en persoon rust dus op de mailbox van de ondertekenaar. Een aparte toegangscode als tweede kanaal is voorbereid in het schema maar niet in gebruik. Wat er wél is als detectiemaatregel: de cliënt krijgt standaard een kopie van het ondertekende stuk, dus bij een vervalsing ziet de echte cliënt dat in zijn mailbox.

9.4 Bestandsverwerking

Uploads worden gecontroleerd op type en grootte. De conversie draait in een eigen tijdelijke map met een privé-profiel, een uitgekleeide omgeving (geen proxyvariabelen, geen tokens, geen databasewachtwoord, geen sleutels), een harde tijdslimiet en een limiet op de uitvoer. Downloads gaan met `Content-Disposition: attachment` en `nosniff`.

Preciezer dan hier eerder stond. Netwerktogang voor die conversieprocessen wordt *niet* afgedwongen. Ze draaien in de webcontainer, en die heeft netwerk nodig voor de sealer, SMTP en de provider-API's. Wat er is: geen proxyvariabelen en niets uit de omgeving. Wie het echt dicht wil, zet de conversie in een eigen container met `network_mode: none`. Dat is een bewuste openstaande keuze.

Een virusscan is beschikbaar maar staat uit: een residente scanner vraagt 1,5 tot 2 GB voor de virusdefinities, de helft van deze machine. Uploads komen alleen van ingelogde medewerkers vanaf kantoormachines met eigen bescherming.

9.5 Integriteit bij downloaden

Bij elke download wordt de vastgelegde hash opnieuw berekend over de bytes die eruit gaan. Wijkt die af, dan wordt de download geblokkeerd en komt er een `INTEGRITEIT_AFWIJKING` in het auditspoor.

9.6 Opstartregels

Alle 88 omgevingsvariabelen worden bij het opstarten gevalideerd; een ontbrekend of te zwak geheim laat de applicatie niet starten. Daarnaast drie harde weigeringen:

- de openbare teststub van de ondertekenprovider in productie — op de omgevingsvlag *en* op het bekende client-id, met witruimte eraf;
- `SEAL_MODE=none` in productie zonder de tweede vlag `ALLOW_UNSEALED` ;
- `SEAL_MODE=qualified` zonder werkende ondertekenprovider.

9.7 Privacy

Alle gegevens in de EU. Dataminimalisatie: van tekenlinks en codes alleen hashes, van het openen van mail bewust géén bewijsvoering (geblokkeerde afbeeldingen missen een opening, en vooraf-ophalende privacybescherming meldt er één die niet plaatsvond). Bewaartermijnen zoals in hoofdstuk 7. Back-ups versleuteld, met de sleutels bewust buiten dezelfde back-up.

E-mail en bezorging

10.1 Drie transporten

`MAIL_TRANSPORT` is `smtp`, `postmark` of `resend`. Bij SMTP weet je alleen dat je de mail aan de server hebt aangeboden. Of hij is afgeleverd, gebounced of in de spamfolder belandde, weet je niet. In een bewijsdossier is “verzonden” daarmee de zwakste schakel; bij de andere twee komt het message-id terug en wordt de bezorgstatus verwerkt.

10.2 Het webhook-eindpunt

Postmark	HTTP-basicauth op de URL, met een optionele lijst toegestane IP-adressen
Resend	Svix-signatuur (HMAC-SHA256 over id, tijdstempel en body) met een tijdvenster van 5 minuten tegen replay
Zonder instellingen	alles wordt geweigerd — geen open eindpunt
Herhaling	idempotent op het gebeurtenis-id van de provider

10.3 Bounces

Een *harde* bounce is definitief: de eigenaar krijgt bericht, herinneren wordt geblokkeerd, en het dossier krijgt een melding op het dashboard. Zonder dat signaal bloedt zo'n dossier stil dood. Een *tijdelijke* bounce levert één nieuwe poging na een uur op; daarna geldt hij als hard. Een “tijdelijke” bounce op een uitgezet adres geldt direct als hard.

Omdat van de tekenlink alleen de hash is bewaard, kan de oorspronkelijke link niet opnieuw worden verstuurd. De ontvanger krijgt een nieuwe uitnodiging met een nieuwe link.

10.4 Herinneringen en vervaltermijn

Herinneringen op **dag 5 en dag 12** na verzending, maar alleen als de link daarna nog minstens een etmaal bruikbaar is. Bij een korte geldigheidsduur valt de tweede dus weg: een link die niet werkt is erger dan stilte. Wie al heeft getekend of een onbezorgbaar adres heeft, krijgt niets.

Mislukt het versturen (mailserver even weg), dan blijft de teller staan en probeert de volgende ronde het opnieuw. Zou de teller altijd oplopen, dan is een uur storing genoeg om een herinnering definitief te laten verdwijnen.

Bij het verstrijken van de geldigheidsduur gaat het dossier op `VERLOPEN`: ongebruikte links worden ingetrokken, de eigenaar krijgt bericht met wie nog niet had getekend, en de al gezette handtekeningen blijven staan als bewijs van wat er is gebeurd. Er wordt niets verzegeld en er gaat geen voltooiingsmail uit.

10.5 Opnieuw versturen — en het verschil met weigeren

Situatie	Wat er kan
VERLOPEN	opnieuw te versturen in hetzelfde dossier: nieuwe links, nieuwe vervaldatum, herinneringen weer op nul. Alleen wie nog niet had getekend krijgt een nieuwe link.
GEWEIGERD	onherroepelijk. Weigeren is een besluit; opnieuw aanbieden betekent een nieuw dossier.

Dat onderscheid is er niet voor de netheid. Zou een verlopen verzoek een nieuw dossier vereisen — documenten opnieuw uploaden, velden opnieuw plaatsen — dan zet iemand binnen een maand de geldigheidsduur op een jaar, en is de vervaltijd feitelijk weg. De praktijk is “de cliënt was op vakantie”.

Achtergrondtaken

Een wachtrij in Postgres. Geen Redis, geen aparte broker.

11.1 Hoe de wachtrij werkt

Taken staan in een tabel. Een worker claimt ze met `SELECT ... FOR UPDATE SKIP LOCKED`, waardoor meerdere workers naast elkaar kunnen draaien zonder dezelfde taak twee keer te pakken. Mislukt een taak, dan komt hij terug met een oplopende tussentijd: 1, 5, 15 en 60 minuten, daarna elk uur. Bij het bereiken van het maximum blijft de taak staan met de laatste foutmelding — hij verdwijnt niet stil.

Terugkerende taken plannen zichzelf opnieuw in, in een `finally`: ook na een fout mag de reeks niet stilvallen. Er is geen aparte cron nodig.

11.2 De taken

Taak	Wanneer	Wat
SEAL_RETRY	na een mislukte verzegeling	opnieuw proberen, max 12 keer, met mail na de derde
REMINDER	elk uur	herinneringen die aan de beurt zijn
EXPIRE	elk uur	verlopen dossiers afsluiten en melden
WAARMERK_NUDGE	elk uur	eigenaar porren na 3 dagen wachten op zijn waarmerk
ARCHIVE_NUDGE	elk uur	porren over afgeronde stukken die nog niet in SharePoint staan (dag 60 en 80)
MAIL_RESEND	een uur na een tijdelijke bounce	één nieuwe uitnodiging
CSC_SESSION_CLEANUP	elke 5 minuten	verlopen ondertekensessies opruimen
ORPHAN_CLEANUP	elke 6 uur	bestanden opruimen waar geen rij naar wijst, ouder dan 24 uur
AUDIT_ANCHOR	dagelijks + per afgerond dossier	de kop van de keten buiten de database vastleggen
RETENTION_CLEANUP	dagelijks	bestanden na 90 dagen, dossiers na 7 jaar

11.3 Waarom die opruimtaak een marge van 24 uur heeft

Elke bewerking schrijft een *nieuw* bestand; de database verplaatst alleen de verwijzing. Daardoor blijft er af en toe een bestand achter waar niets naar wijst: een transactie die terugrolde, een verlopen ondertekensessie, een verwijdering die na de commit mislukte. Die zijn veilig te verwijderen, met één harde voorwaarde: alleen als ze ouder zijn dan 24 uur. Zonder marge ruimt de taak precies het bestand op dat net is geschreven en waarvan de commit nog loopt. De taak logt eerst wat hij zou verwijderen en doet het daarna.

Documentherkenning

Bedoeld om typewerk te schelen, niet om beslissingen te nemen. Alles wat de herkenning voorstelt, is te overschrijven.

12.1 De keten

1. **Tekstlaag.** Snel en foutloos voor digitale PDF's, bijvoorbeeld uit Visionplanner.
2. **OCR als terugval.** Is de tekstlaag (vrijwel) leeg, dan gaat het document via poppler naar afbeeldingen en via tesseract met Nederlandse taaldata naar tekst. Alleen de eerste paar pagina's: meer is voor classificatie onnodig.
3. **Trefwoorden** bepalen het type, een reguliere expressie het boekjaar.
4. **Sjabloon.** Op basis van type en jaar wordt een titel en een begeleidend bericht voorgesteld, met invulvelden voor de voornaam van de cliënt, de bedrijfsnaam, het boekjaar en de afzender.

Worden een jaarrekening, notulen en een bevestiging samen geüpload, dan stelt de applicatie één gecombineerd verzoek voor — dat is in de praktijk vrijwel altijd wat er moet gebeuren.

Er gaat geen data naar buiten: zowel de tekstextractie als de OCR gebeuren lokaal.

12.2 Velden plaatsen

De pagina's worden in de browser gerenderd met pdfjs. De medewerker tekent rechthoeken; die worden van pixels omgerekend naar PDF-punten met de oorsprong linksonder, precies zoals de stempelfunctie ze verwacht. Per vak staat vast voor wie het is: een eigen veld (direct te stempelen met de opgeslagen handtekening) of een veld voor een ontvanger.

Gelijktijdigheid en vergrendeling

Dit hoofdstuk bestaat omdat hier de ernstigste fouten zaten. Ze gaven geen van alle een foutmelding.

13.1 Twee ondertekenaars tegelijk

Bij gelijktijdig ondertekenen kunnen twee cliënten op hetzelfde moment indienen. Het stempelen was een lees-bewerk-schrijf op hetzelfde document: beiden lazen dezelfde versie, stempelden hun eigen handtekening erop, en de laatste schreef de ander weg. De database zei dat beide velden waren gevuld en beide ondertekenaars stonden op “ondertekend”, maar in het bestand stond één handtekening.

Nu zit de hele bewerking — alle documenten, de status, de tekenlink én de auditregel — in één transactie met een rijvergrendeling per document. Gelijktijdige indieningen wachten netjes op elkaar en elke stempel komt op het resultaat van de vorige.

13.2 Dezelfde fout in het auditspoor

Bij het testen van het bovenstaande bleek de hashketen te vorken: twee gelijktijdige schrijvers lazen dezelfde laatste regel en verwezen beide naar dezelfde voorganger. Dat is **niet te herstellen** (de regels zijn append-only) en de controle zou daarna voor altijd een breuk melden die niemand had veroorzaakt. Een controle die altijd rood staat, wordt genegeerd — en dan beschermt de hele keten niets meer.

Opgelost met een advisory lock per keten: één schrijver tegelijk.

13.3 De globale vergrendelingsvolgorde

Meerdere vergrendelingen in verschillende ordes leidt tot deadlock. Daarom staat de volgorde op één plek vastgelegd, met de paden die er meer dan één nemen erbij:

1. de dossierrij;
2. de documentrijen, oplopend op id;
3. de advisory ketenlocks, en daarbinnen: dossierketens oplopend, de dossierloze keten altijd als laatste.

Dat derde niveau heeft een subvolgorde nodig omdat het opruimen na de bewaartermijn er twee kan raken. In de implementatie wordt de grafsteen na de commit geschreven, dus die twee worden nooit tegelijk vastgehouden.

13.4 Wachten met een grens

Er wordt een rijvergrendeling vastgehouden terwijl er een PDF wordt geladen, gestempeld, versleuteld en weggeschreven. Bij een jaarrekening met veel afbeeldingen zijn dat seconden. Zonder grens hangt het verzoek tot de proxy het afkapt, en dan weet de ondertekenaar niet of het is gelukt.

Daarom een limiet van tien seconden op het wachten en een ruimere grens op de bewerking zelf. Loopt die af, dan krijgt de ondertekenaar een nette, herhaalbare melding en is zijn tekenlink nog geldig — die zit in dezelfde transactie, dus een terugrol geeft hem terug.

13.5 Nooit een opslagsleutel overschrijven

Elke schrijfactie maakt een *nieuw* bestand met een willekeurige naam; de transactie verplaatst alleen de verwijzing, en het opruimen van de oude versie gebeurt *na* de commit. Zou dat ervoor gebeuren, dan is bij een terugrol de oude versie weg terwijl de database er nog naar wijst: dataverlies, de kant die niet mag.

13.6 Eén zware bewerking tegelijk

LibreOffice en tesseract vragen elk honderden megabytes. Op 4 GB is twee van die twee tegelijk de manier waarop de server omvalt. Ze draaien daarom onder één advisory lock in Postgres: nooit meer dan één tegelijk, *over containergrenzen heen*. Een semafoor in het geheugen zou hier niet volstaan — web en worker zijn aparte processen.

De tweede medewerker wacht dus even; bij deze volumes gaat dat om seconden, en na twee minuten volgt een leesbare melding in plaats van een omgevallen server.

Back-up en herstel

14.1 Wat het kroonjuweel is

Dat is met het nieuwe bewaarmodel verschoven. De documentbestanden staan er maar 90 dagen en het archief is SharePoint. De **database** is nu het belangrijkste: daar staat zeven jaar auditspoor met alle hashes en metadata.

Wat	Hoe	Waarom
Postgres	dagelijks volledig, versleuteld	zeven jaar bewijs
Documentenvolume	wekelijks incrementeel	staat er maar 90 dagen
Sleutels en <code>.env</code>	apart, buiten dezelfde back-up	anders back-up je het slot met de sleutel erin

14.2 Naar buiten de server

Een back-up die op dezelfde machine staat is geen back-up. Er is een script voor restic: gededupliceerd, incrementeel, versleuteld naar externe opslag (bijvoorbeeld een Storage Box). Versleutelen gebeurt vóór het uploaden, dus de jurisdictie van de opslagpartij doet praktisch niet mee — die ziet onleesbare blokken. Bewaard worden 14 dagelijkse, 8 wekelijkse, 24 maandelijkse en 7 jaarlijkse momentopnamen; elke run controleert de structuur plus 5% van de blokken.

14.3 De restoretest

Een back-up die je nooit hebt teruggezet, is geen back-up. Er is een script dat de back-up in een weggooi-omgeving terugzet en daarna controleert of er echt iets bruikbaars uit komt:

- staan er dossiers en een auditspoor in?
- is de hashketen intact?
- zijn de documenten te ontsleutelen en klopt hun hash?
- valideert een ondertekend document nog?
- **zijn alle sleutelversies waar data naar verwijst nog beschikbaar?**

Dat laatste is de controle die de rest waardeloos maakt als hij ontbreekt. Een restore van drie jaar oud is versleuteld met de sleutel van toen. Staat die niet meer in de omgeving, dan komt er wel een database uit de back-up maar geen leesbaar document — en dat merk je zonder deze controle pas terwijl een cliënt wacht.

De productieomgeving wordt niet aangeraakt en alles wordt aan het eind opgeruimd. Leg de uitkomst vast in het kwaliteitshandboek: datum, wie het deed, en of het slaagde.

Kwaliteit en tests

15.1 221 automatische controles

Pakket	Aantal	Wat het dekt
Opstartregels	11	de harde weigeringen bij het opstarten
Labels en modus	13	ondertekenen versus verzegelen per documentsoort
Auditspoor	18	hashketen, append-only trigger, certificaat, opslagversleuteling
Gelijktijdig ondertekenen	19	twee indieningen op hetzelfde moment, en de inhoud van het ondertekencertificaat
Ondertekensessie	8	versleuteld bewaren en wissen, eigenaarscontrole
Levensloop	32	herinneren, verlopen, opnieuw verzenden, nudge
Opslag en bewaartermijn	45	nieuwe sleutels, weesbestanden, het bewaarmodel, de grendel
Sealer	26	verzegelen, idempotentie, manipulatie detectie, validatie
Certificering	16	DocMDP P=1, de DSS-vraag, en wat een viewer wél en niet blokkeert
Herverificatie en download	24	verse TOTP-code, replaybescherming, gescheiden downloadtoken
Validatormodus	4	de omgekeerde grendel in het juiste proces

Ze draaien tegen een echte PostgreSQL en een echte pyHanko, niet tegen nagemaakte onderdelen. De sealertests gebruiken een zelf aangemaakt testcertificaat.

15.2 Van de reparaties is bewezen dat de test ze vangt

Een test die groen staat bewijst niets als hij ook groen zou staan zonder de reparatie. Van beide vergrendelingen is de code tijdelijk uitgeschakeld en is vastgesteld dat de test dan faalt — bij het stempelen verdwijnt dan de eerste handtekening, bij het auditspoor vorkt de keten. Daarna weer aan, en groen.

Hetzelfde is gedaan met het ondertekencertificaat: door het IP en de user-agent weer op `null` te zetten, precies zoals de oorspronkelijke fout, vallen er vier controles om. Juist bij die fout is dat belangrijk, want het certificaat zág er correct uit — er stond alleen een streepje waar een IP-adres hoorde.

15.3 Wat er verder is aangetoond

- Een verzegeld document is intact, geldig, dekt het hele bestand, en één omgeklapte byte wordt afgekeurd.
- Hetzelfde bestand twee keer aanbieden levert één handtekening en een `409`.
- Een document dat is gewijzigd en opnieuw ondertekend met een zelfgemaakt certificaat komt op de controlepagina uit als “ongewijzigd, uitgever niet te verifiëren” — nergens groen.
- Een kapotte PDF geeft een nette 400, geen onbehandelde serverfout.
- Het bewaarmodel: 200 dagen zonder vinkje houdt alles; vinkje plus 91 dagen wist de bestanden terwijl de hashketen intact blijft.

- Een verplichte auditregel rolt de hele transactie terug.

15.4 De pijplijn

Er is een CI-workflow die op een schone database de migraties toepast, alle controles draait en de productiebuild maakt. Die start voorlopig alleen handmatig: in dit account is geen uitvoeromgeving beschikbaar, en met automatische triggers zou elke wijziging een faalmail opleveren. Een pijplijn die altijd rood staat wordt genegeerd. Zodra er een runner is, zijn het twee regels commentaar die weg moeten.

Het eerlijke gat. Er draait dus nog niets automatisch bij een wijziging. En van de dekking mist één saai maar belangrijk stuk: autorisatie per route (niet ingelogd, ingelogd maar niet de eigenaar, eigenaar). Dat is precies het gat waar na een verbouwing een lek terugkomt.

Hosting en beheer

16.1 De machine

Eén VPS in de EU: 2 vCPU, 4 GB RAM, 40 GB SSD. Dat is genoeg voor dit volume, maar niet met marge voor onzorgvuldigheid. Drie maatregelen maken het veilig door ontwerp in plaats van door geluk:

- zware documentbewerkingen één tegelijk (hoofdstuk 13.6);
- geen residente virusscanner (die zou de helft van het geheugen vragen);
- images bij voorkeur in de CI bouwen, niet op de server — een frontendbuild vraagt 2 tot 4 GB náast de draaiende containers.

16.2 Uitrollen

Er is een deployscript dat de code bijwerkt, images ophaalt (of bouwt, met waarschuwing), de containers herstart, controleert of de migraties zijn gelopen, en **oude images opruimt met behoud van twee generaties**. Dat laatste is bij 40 GB het enige dat de schijf realistisch laat vollopen, en een volle schijf betekent geen uploads en geen back-up meer. Het script waarschuwt ook als er minder dan 5 GB vrij is.

16.3 Wat een beheerder moet weten

Auditspoor controleren	één commando; meldt of de keten intact is en anders precies waar hij breekt
Bewaartermijn	eerst een droogloop, dan uitvoeren
Sleutelrotatie	drie stappen zonder downtime; oude sleutel pas weg als het script meldt dat niets er meer naar verwijst
Volle schijf	hoe je het herkent en wat je dan doet
Regressietests	één commando, tegen een testdatabase — nooit tegen productie

Dat staat allemaal in `docs/beheer.md`, met bij elke maatregel de grens erbij in plaats van alleen de belofte.

16.4 Kosten per jaar

Post	Bedrag
VPS (2 vCPU, 4 GB, 40 GB) in een Nederlands datacenter	± €102
Externe back-upopslag, 1 TB	± €38
Beroepscertificaat, één gebruiker	± €50
Transactionele mail	€0 tot €60
Totaal	± €190 tot €250

Ter vergelijking: een commercieel platform met drie gebruikers zit rond de €950 per jaar, en per-ondertekening-modellen komen bij enkele duizenden stukken hoger uit. Maar de echte kosten zitten niet in de rekening: patchen, monitoren, en meebewegen als de ondertekenprovider of Microsoft iets wijzigt. Dat is de reden om de functielijst gesloten te houden.

HOOFDSTUK 17

Wat bewust niet is gebouwd

Dit is geen vergeetlijst. Bij elk punt staat de reden, want een keuze zonder reden is over een jaar niet meer te verdedigen.

Niet gebouwd	Waarom niet
Een apart organisatiecertificaat en de bijbehorende driver	Het beroepscertificaat doet hetzelfde werk. Eén mechanisme, en enkele honderden euro's per jaar minder.
Een sleutelgebaseerde auditketen (HMAC)	Het gat is echt: met een kale hash kan iemand met alleen databasetoegang de keten consistent herschrijven. Maar het primaire bewijs is de ondertekende PDF, en die kan diegene niet aanraken. Een sleutelversie per regel, sleutels die zeven jaar bewaard moeten blijven en buiten de back-up, plus een controleur die per regel de juiste sleutel kiest — dat is te veel machinerie om aanvullend detail te beschermen. Wat er in plaats daarvan is: de ankers uit hoofdstuk 8.4.
Een aparte container voor de controlepagina	Vervangen door die pagina achter de login te zetten. Geen onbeauthenteerde ingang meer, en een container minder op 4 GB. De rol bestaat nog in de image als de pagina ooit publiek moet worden.
Automatisch archiveren naar SharePoint	De driver staat er, maar staat uit. Handmatig uploaden plus een vinkje is voor tien stukken per dag eenvoudiger dan een koppeling die kan falen. Zodra het stapje gaat irriteren, kan hij aan.
Een aparte toegangscode als tweede kanaal	Besluit van het kantoor. De velden staan klaar in het schema, er is geen gebruikersinterface en geen handhaving. De erkende beperking staat in hoofdstuk 9.3.
Koppelingen met Visionplanner, AFAS of Exact	Documenten komen via upload binnen.
Een aparte status “verzegeld”	“Ondertekend” blijft de eindstatus; het onderscheid zit in <code>sealedStage</code> per document. Een extra status zou “klaar” over twee waarden splitsen.
Periodiek hervernieuwen van tijdstempels	Bij zeven jaar bewaartermijn en ingebedde revocatiegegevens is het document self-contained. Hervernieuwen beschermt tegen cryptografische verzwakking over decennia.
Netwerkisolatie voor de conversieprocessen	Zou een eigen container vragen. Wat er is, staat in hoofdstuk 9.4; de bewering is bijgesteld naar wat er werkelijk wordt afgedwongen.
Juridische kwalificaties in de interface	Er staat “digitaal ondertekend” en “verzegeld”, niet “eIDAS-niveau X” — behalve bij een echte gekwalificeerde handtekening. Bewust geen overclaim.
Een tweede ondertekendriver die namens de accountant tekent	Was er (<code>digidentity</code>), is verwijderd. Hij tekende best-effort en liet een dossier doorlopen naar ONDERTEKEND zonder gekwalificeerde handtekening als de provider wegviel. Zie 6.3.
Een geavanceerde handtekening (AES) voor de cliënt	Zou per ondertekenaar een certificaat op naam bij een TSP vragen, met identificatie via iDIN of een ID-scan vóór elke handtekening — kosten en wrijving per stuk. Voor akkoordbrieven en opdrachtbevestigingen volstaat de huidige methode. Pas bouwen als een tegenpartij het concreet eist, en dan voor dat ene documenttype.

Wat nog niet bewezen is

Het verschil tussen “gebouwd en getest” en “in gebruik bewezen”. Dit hoofdstuk hoort in dit verslag omdat de rest anders sterker klinkt dan het is.

18.1 De ondertekenprovider is nog nooit echt gesproken

Alle netwerkaanroepen naar de ondertekenprovider zijn gebouwd volgens hun documentatie en zijn niet tegen de echte dienst uitgevoerd — de ontwikkelomgeving mag daar niet bij. Twee punten waarover hun documentatie niet eenduidig is, staan open: hoe precies het servicetoken wordt opgehaald, en hoe een verlopen autorisatietoken eruitziet. De PDF-kant van het tweefasige ondertekenen is wél end-to-end bewezen, met een zelf aangemaakt testcertificaat.

18.2 Er is nog geen certificaat en geen tijdstempeldienst

De applicatie draait daarom met de verzegeling uit. Dat is een geldige stand voor een pilot — de banner, de certificaatregel en de auditregel maken het zichtbaar — maar het betekent dat de kern van hoofdstuk 6 alleen met een testcertificaat is aangetoond.

18.3 De volledige stapel is nooit als geheel opgestart

Alle onderdelen zijn apart getest en de applicatie is los gedraaid, maar `docker compose up` met alle vijf de services samen is niet uitgevoerd. Dat is het grootste onbekende: één verkeerde variabele, een healthcheck die niet slaagt, of Caddy dat geen certificaat krijgt. Ook is de volledige cliëntreis na de laatste wijzigingen niet als mens doorlopen.

De aanbeveling. Een generale repetitie op de echte server met eigen mensen als “cliënt”: één dossier met twee ondertekenaars, één weigering, en één dat je laat verlopen en opnieuw verstuurt. Daarna één keer back-up maken en terugzetten, vóórdat er echte cliëntgegevens in staan.

18.4 De mailprovider is nog niet gekozen

SMTP werkt. De verwerking van bezorgstatussen is getest met nagmaakte berichten van beide providers, niet met echte. Zolang er SMTP wordt gebruikt, is “verzonden” de zwakste schakel in het bewijsdossier.

18.5 Er draait geen pijplijn

De 221 controles draaien handmatig. Dat is het grootste procesgat: handmatige tests verlopen binnen enkele maanden zonder dat iemand het merkt.

18.6 Eén openstaande vraag voor het kantoor

Wordt er met de pen op het aangiftestuk zelf ondertekend, of alleen op de akkoordbrief daarbij? Dat bepaalt of de akkoordverklaringen “Ondertekend door <naam>, RA” of “Verzegeld door Otto Visser & Partners Accountants” krijgen. Het antwoord is een instelling, geen wijziging in de code, dus het kan later komen.

18.7 Tot slot

Wat hier staat is met opzet uitgesplitst tussen wat werkt, wat is aangetoond, en wat nog moet blijken. Bij elke maatregel in dit verslag staat de grens erbij, omdat een maatregel waarvan iemand denkt dat hij

meer doet dan hij doet gevaarlijker is dan geen maatregel. Dat geldt voor de append-only trigger, voor de sandbox rond de conversie, voor de ankers, en voor de controlepagina.